

Laboratório de Banco de Dados

Profa. Msc. Esteic Janaina
Santos Batista

Módulo 3 – Introdução a Conceitos de Processamento de Transações e Controle de Concorrência

Unidade 2 - Mapeamento OR e Conexão com Aplicação com Banco de Dados



UFMS

AGEAD

Agência de Educação
Digital e a Distância

Conexão com banco

- A conexão de uma aplicação com um banco de dados é o processo que permite que uma aplicação interaja com um banco de dados para realizar operações, como inserir, consultar, atualizar ou deletar dados.
- Em termos práticos, ela é a ponte que permite que o código da aplicação troque informações com um banco de dados relacional ou não-relacional.

Conexão com banco: JDBC

- **JDBC (Java Database Connectivity)** é uma API de baixo nível para conexão com bancos de dados em Java para executar comandos SQL.
- Embora seja muito poderoso, o JDBC requer que os desenvolvedores **gerenciem manualmente as conexões**, preparem consultas e lidem com o tratamento de exceções de baixo nível.
- O JDBC fornece uma interface direta para o banco de dados, permitindo abrir conexões, executar comandos SQL (como SELECT, INSERT, UPDATE) e recuperar resultados.

Conexão com banco: JDBC

demo2 > src > main > java > com > example > J App.java > App

```
13 public class App
    Run | Debug
15 public static void main( String[] args )
16 {
17     try{
18         Connection conexao = DriverManager.getConnection(url:"jdbc:postgresql://localhost:5432/Aulas", user:"postgres", password:"123root");
19         if(conexao != null){
20             System.out.println(x:"Banco de dados conectado com sucesso");
21             Statement stm = conexao.createStatement();
22             // Chama a função consultaDados para buscar os dados
23             consultaDados(stm);
24             stm.close();
25             conexao.close();
26         }else{
27             System.out.println(x:"Conexão falhou!");
28         }
29     }catch (SQLException e){
30         e.printStackTrace();
31     }
32 }
33
34 public static void consultaDados(Statement stm){
35     String query = "SELECT * FROM alunos;";
36     try {
37         ResultSet result = stm.executeQuery(query);
38         while (result.next()) {
39             System.out.println(result.getString(columnLabel:"nome"));
40         }
41     } catch (Exception e) {
42         e.printStackTrace();
43     }
44 }
```

Exemplo de Conexão JDBC com Java

Estabelecer a conexão

- Para estabelecer a conexão, você precisa da URL do banco de dados, nome de usuário e senha.

```
Connection conn =
```

```
DriverManager.getConnection("jdbc:postgresql://localhost:5432/meuBanco", "usuario", "senha");
```

Exemplo de Conexão JDBC com Java

Criar uma instrução (Statement)

- Um Statement é usado para executar consultas e comandos SQL

```
Statement stmt = conn.createStatement();
```

Exemplo de Conexão JDBC com Java

Executar uma consulta

- Você pode executar consultas SQL como SELECT, INSERT, UPDATE e DELETE usando o objeto Statement.
- Exemplo de um comando SQL para recuperar dados:

```
ResultSet rs = stmt.executeQuery("SELECT * FROM clientes");
```

```
stmt.executeUpdate("INSERT INTO clientes (nome, email)  
VALUES ('João', 'joao@email.com')");
```


Exemplo de Conexão JDBC com Java

Processar os resultados

Você pode processar os dados do ResultSet para exibi-los ou manipulá-los.

```
while (rs.next()) {  
    String nome = rs.getString("nome");  
    String email = rs.getString("email");  
    System.out.println("Nome: " + nome + ", Email: " + email);  
}
```

Exemplo de Conexão JDBC com Java

Fechar a conexão

- Após terminar as operações com o banco de dados, é importante fechar a conexão para liberar recursos.

```
rs.close();
```

```
stmt.close();
```

```
conn.close();
```

Exemplo de Conexão JDBC com Java

Resumo de como o JDBC funciona:

- **Connection:** é o objeto responsável por gerenciar a conexão com o banco de dados.
- **Statement ou PreparedStatement:** são objetos usados para enviar consultas SQL ao banco de dados.
- **ResultSet:** armazena os resultados das consultas SQL (usado para SELECT).

Mapeamento Objeto-Relacional (OR)

- O Mapeamento Objeto-Relacional (OR), ou ORM (*Object-Relational Mapping*), é uma técnica utilizada para converter dados entre sistemas de tipos incompatíveis usando orientação a objetos.
- Ele cria uma "ponte" entre o mundo dos objetos nas linguagens de programação (como Java) e o mundo dos bancos de dados relacionais (como PostgreSQL ou MySQL).

Mapeamento Objeto-Relacional (OR)

- No Java, você trabalha com objetos, enquanto os bancos de dados utilizam tabelas.
- O Mapeamento OR facilita a tarefa de transformar (ou mapear) essas classes e objetos em tabelas e registros de forma automática, sem que você tenha que escrever todo o código SQL manualmente.
- Isso economiza tempo, reduz erros e mantém o foco do desenvolvimento no código orientado a objetos.

Por que usar Mapeamento OR?



MS DIGITAL

AGEAD
Agência de Educação
Digital e a Distância



- **Automatiza a interação com o banco de dados:** Você pode trabalhar diretamente com objetos no seu código Java, e o Mapeamento OR se encarrega de traduzir as operações com esses objetos para o banco de dados.
- **Reduz a necessidade de SQL manual:** A maioria das operações SQL (inserir, atualizar, deletar) pode ser feita através do código Java.
- **Facilita a manutenção:** Ao mapear as entidades (classes Java) diretamente para as tabelas, o código fica mais organizado e fácil de manter.

Relacionamento entre as classes

- Além de mapear as classes para tabelas, o Mapeamento OR também permite que você defina relacionamentos entre as classes, como no banco de dados.
- Por exemplo, você pode ter uma relação entre Cliente e Produto, onde um cliente pode comprar vários produtos.
- Com a anotação `@OneToMany`, definimos que um cliente pode comprar vários produtos e o Mapeamento OR vai gerenciar isso, criando uma tabela de relacionamento no banco de dados para armazenar essas informações.

Relacionamento entre as classes

- @OneToOne (Um-para-Um)

- Define um relacionamento de "Um-para-Um" entre duas entidades. Isso significa que uma instância de uma entidade está associada a no máximo uma instância de outra entidade.

- @ManyToMany (Muitos-para-Muitos)

- Define um relacionamento "Muitos-para-Muitos", onde várias instâncias de uma entidade estão associadas a várias instâncias de outra entidade. Esse tipo de relacionamento geralmente envolve uma tabela intermediária.

Relacionamento entre as classes

@Entity

```
public class Cliente {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String nome;
```

```
    @OneToMany
```

```
    private List<Produto> produtosComprados;
```

```
    // Construtores, Getters e Setters
```

```
}
```

Benefícios práticos

- Você não precisa escrever comandos SQL para inserir, atualizar, ou deletar registros. Tudo isso é feito através das operações normais com os objetos Java.
- A estrutura do banco de dados fica "escondida", e você pode focar no código orientado a objetos, enquanto o Mapeamento OR cuida de como esses objetos se relacionam com o banco de dados.

Benefícios práticos

- Exemplo: Se você quiser adicionar um novo cliente ao banco de dados, basta criar uma instância da classe Cliente e salvar:

```
Cliente cliente = new Cliente();
```

```
cliente.setNome("João");
```

```
cliente.setEmail("joao@gmail.com");
```

```
// A linha abaixo insere o cliente no banco de dados
```

```
entityManager.persist(cliente);
```

Conexão com banco

- Conectar um aplicativo Java a um banco de dados envolve várias ferramentas e tecnologias, cada uma desempenhando um papel específico.

Conexão com banco: Maven

- Maven é uma ferramenta de automação de build e gerenciamento de dependências para projetos Java. Sua principal função é facilitar o processo de construção, gerenciamento e integração de bibliotecas em projetos Java.
- Maven baixa e gerencia automaticamente as bibliotecas necessárias para o projeto, como as bibliotecas de JPA, Hibernate, e JDBC. Ao invés de adicionar manualmente essas bibliotecas ao projeto, o Maven permite que você defina todas as dependências em um arquivo chamado pom.xml.

Conexão com banco: Maven

```
<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.11</version>
    <scope>test</scope>
  </dependency>

  <!-- https://mvnrepository.com/artifact/org.postgresql/postgresql -->
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <version>42.7.4</version>
  </dependency>

  <!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-core -->
  <dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.6.1.Final</version>
    <type>pom</type>
  </dependency>

  <!-- https://mvnrepository.com/artifact/org.hibernate/hibernate-entitymanager -->
  <dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-entitymanager</artifactId>
    <version>5.6.15.Final</version>
  </dependency>
</dependencies>
```

Conexão com banco: JPA

- JPA (Java Persistence API) é a especificação oficial da Java EE para o mapeamento objeto-relacional (ORM). Ele define um conjunto de regras para gerenciar persistência de dados em bancos de dados relacionais através de objetos Java.
- Permite que você defina entidades Java (classes) que correspondem a tabelas no banco de dados (ORM).
- O JPA abstrai a camada de persistência, permitindo que os desenvolvedores manipulem os dados no banco de dados como objetos Java, sem precisar escrever consultas SQL diretamente.

Conexão com banco: JPA

```
@Entity
public class Aluno implements Serializable {
    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false) // Define que o campo 'nome' é NOT NULL
    private String nome;

    @Temporal(TemporalType.DATE)
    @Column(nullable = false) // Define que o campo 'dataNascimento' é NOT NULL
    private Date dataNascimento;

    @Column(nullable = false, unique = true) // Define que o campo 'email' é NOT NULL e UNIQUE
    private String email;
```


Conexão com banco: Hibernate

- **Hibernate** é uma das bibliotecas mais populares que **implementa a especificação JPA**.
- O Hibernate transforma as instruções JPA em consultas SQL que interagem diretamente com o banco de dados.
- Ele lida com a criação e gerenciamento de sessões (conexões) com o banco de dados e fornece um EntityManager para executar operações CRUD.

Exemplo prático

- Imagine o sistema de matrícula de alunos dos módulos anteriores.
- Em vez de escrever SQL diretamente, você pode criar classes Java que representam essas entidades.
- Veja o exemplo de um Mapeamento Objeto-Relacional (ORM) utilizando JPA (Java Persistence API).

Exemplo prático

- **@Entity**: Essa anotação define que a classe Cliente é uma entidade do banco de dados. No contexto do ORM (Object-Relational Mapping), ela indica que essa classe será mapeada para uma tabela em um banco de dados relacional. O JPA gerenciará automaticamente a correspondência entre os objetos Java e os registros na tabela.

Exemplo prático

- **@Id:** define que o campo id será a chave primária (primary key) da tabela no banco de dados. Isso significa que cada instância da classe Cliente terá um identificador exclusivo, que será representado por este campo.

Exemplo prático

- `@GeneratedValue(strategy = GenerationType.IDENTITY)`: Esta anotação indica que o valor do campo id será gerado automaticamente pelo banco de dados. O atributo `strategy = GenerationType.IDENTITY` especifica que o valor será gerado usando uma estratégia de identidade, o que significa que o banco de dados gerará um novo valor para cada nova entrada (registro) automaticamente.

Referências



POSTGRESQL. **Tradução da Documentação para o Português do Brasil.** Disponível em: <https://link.ufms.br/Yk4kw> Acesso em: 03 maio 2024. **Trecho a ser lido: Capítulo 6.**

SILBERSCHATZ, Abraham; Korth, Henry F.; Sudarshan, S. **Sistema de Banco de Dados.** Rio de Janeiro: GEN LTC, 2020. ISBN 9788595157552. **Disponível na Biblioteca Digital da UFMS.** **Trecho a ser lido: Capítulo 24.**

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](#).

